python exception class example

Python Exception Class Examples

1. Built-in Exception (Basic)

```python
try:
    x = 1 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
```

2. Custom Exception Class (Simple)

```python
class MyError(Exception):
    pass

try:
    raise MyError("Something went wrong")
except MyError as e:
    print(f"Caught: {e}")
```

3. Custom Exception with Extra Data

```python
class ValidationError(Exception):
    def __init__(self, field, message):
        self.field = field
        self.message = message
        super().__init__(f"{field}: {message}")

try:
    raise ValidationError("email", "invalid format")
except ValidationError as e:
    print(e)          # email: invalid format
    print(e.field)    # email
```

```
    print(e.message)  # invalid format
```

4. Exception Hierarchy

python

```
class AppError(Exception):
    """Base class for app errors."""
    pass

class DatabaseError(AppError):
    pass

class NetworkError(AppError):
    pass

# Catch specific
try:
    raise DatabaseError("connection failed")
except DatabaseError as e:
    print(f"DB issue: {e}")

# Catch all app errors
try:
    raise NetworkError("timeout")
except AppError as e:
    print(f"App error: {e}")
```

5. Full Example with Error Codes

python

```
class APIError(Exception):
    def __init__(self, code, message, status=400):
        self.code = code
        self.message = message
        self.status = status
        super().__init__(message)

    def to_dict(self):
        return {"code": self.code, "message": self.message}


class NotFoundError(APIError):
```

```
class NotFoundError(APIError):
    def __init__(self, resource):
        super().__init__(
            code="NOT_FOUND",
            message=f"{resource} not found",
            status=404,
        )


def get_user(user_id):
    if user_id not in {1, 2, 3}:
        raise NotFoundError("User")
    return {"id": user_id}


try:
    get_user(99)
except NotFoundError as e:
    print(e.to_dict())   # {'code': 'NOT_FOUND', 'message': 'User not found'}
    print(e.status)      # 404
```

Key points

- **Always inherit from** `Exception` (not `BaseException`) for normal errors.
- **Call** `super().__init__(...)` so `str(e)` works properly.
- **Use a base exception class** for your app/library to allow catching everything at once.
- **Avoid bare** `raise` unless re-raising inside an `except` block.

Want an example for a specific scenario (e.g., retry logic, context managers, or exception chaining with `raise ... from ...`)?

So if I pass message to super(), if I print(ex) it will print the message

Yes, exactly. ✅

When you call `super().__init__(message)`, the message is passed to `Exception.__init__`, which stores it in `args`. That's what `__str__` returns when you `print(e)` or `str(e)`.

```
python
```

```python
class MyError(Exception):
    def __init__(self, message):
        super().__init__(message)

try:
    raise MyError("boom")
except MyError as e:
    print(e)        # boom
    print(str(e))   # boom
    print(repr(e))  # MyError('boom')
    print(e.args)   # ('boom',)
```

The mechanism

`Exception.__str__` behavior:

- **1 arg** → returns `str(args[0])`
- **0 args** → returns `''`
- **2+ args** → returns `str(args)` (the tuple)

python

```python
print(str(Exception()))              # ''
print(str(Exception("boom")))        # boom
print(str(Exception("a", "b")))      # ('a', 'b')
```

If you forget `super().__init__()`

python

```python
class BadError(Exception):
    def __init__(self, message):
        self.message = message
        # no super() call

try:
    raise BadError("boom")
except BadError as e:
    print(e)         # ''  ← empty!
    print(e.args)    # ()  ← empty
    print(e.message) # boom (only via attribute)
```

So `super().__init__(message)` is what makes `print(e)` show the message.

Multiple values

If you want `print(e)` to show something clean while still passing multiple values, override `__str__`:

```python
class ValidationError(Exception):
    def __init__(self, field, message):
        self.field = field
        self.message = message
        super().__init__(f"{field}: {message}")

try:
    raise ValidationError("email", "invalid")
except ValidationError as e:
    print(e)        # email: invalid
    print(e.args)   # ('email: invalid',)
```

Or keep raw args and customize `__str__`:

```python
class ValidationError(Exception):
    def __init__(self, field, message):
        self.field = field
        self.message = message
        super().__init__(field, message)

    def __str__(self):
        return f"{self.field}: {self.message}"

e = ValidationError("email", "invalid")
print(e)        # email: invalid
print(e.args)   # ('email', 'invalid')
```

Rule of thumb: pass a single human-readable string to `super().__init__()` and store any structured data as extra attributes.